KTH Royal Institute of Technology

SF2527 Numerical Methods for Differential Equations I

Computer Exercise 1

Ordinary Differential Equations and Elliptic Equations

Exercise number: Computer Exercise 1
Group: SF2527 CE1 1

Student	Name	Personal number	Email
1	Lukas Sönnergren	____________	__________________
2	Tage Sköldefors	____________	__________________

HT26

August 29, 2026

Contents

1 Runge–Kutta accuracy and stability

The Runge–Kutta method used in Parts 1 and 2 is

$$k_1 = f(t_n, u_n), \qquad k_2 = f(t_n + h, u_n + hk_1), \tag{1.1}$$

$$k_3 = f\left(t_n + \frac{h}{2}, u_n + \frac{h}{4}k_1 + \frac{h}{4}k_2\right), \qquad u_{n+1} = u_n + \frac{h}{6}(k_1 + k_2 + 4k_3). \tag{1.2}$$

It is explicit, has three stages and uses one step at a time.

1.1 Landau–Lifshitz system

Let

$$a = \frac{1}{4}\begin{bmatrix} 1 \\ \sqrt{11} \\ 2 \end{bmatrix}, \qquad C = \frac{1}{4}\begin{bmatrix} 0 & -2 & \sqrt{11} \\ 2 & 0 & -1 \\ -\sqrt{11} & 1 & 0 \end{bmatrix}.$$

Since $Cm = a \times m$, the ODE is

$$m' = Am, \qquad A = C + \alpha C^2, \qquad \alpha = 0.07, \qquad m(0) = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^{\mathsf{T}}.$$

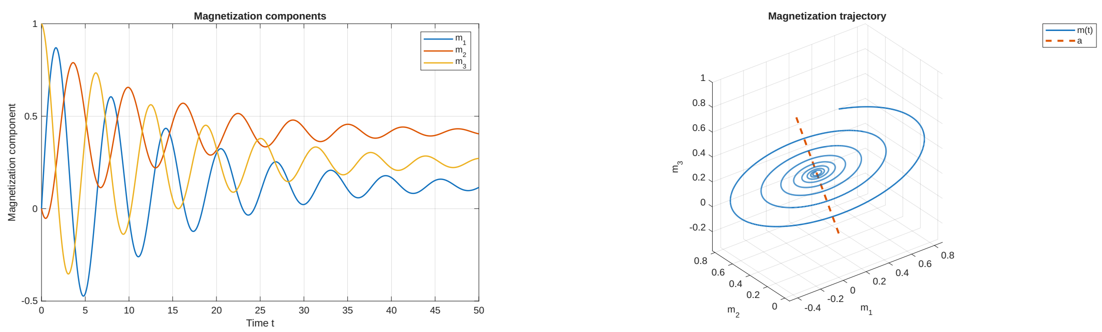

Figure 1: Magnetization components and trajectory for $0 \le t \le 50$ with $h = 0.01$.

The value $a^{\mathsf{T}}m(t) = a^{\mathsf{T}}m(0) = 1/2$ stays constant. Therefore $m(t)$ stays in a plane perpendicular to a. The part perpendicular to a goes to zero, so

$$m(t) \longrightarrow (a^{\mathsf{T}}m(0))a = \frac{1}{2}a,$$

This is also what we see in the plots.

1.2 Order of accuracy

For $N = 50, 100, 200, 400, 800, 1600$, let $h = 50/N$ and

$$d_N = \|\widetilde{m}_N(50) - \widetilde{m}_{2N}(50)\|_2 .$$

The slope in the log–log plot is

$$\boxed{p = 2.994893},$$

which is close to 3. The method is therefore third order.

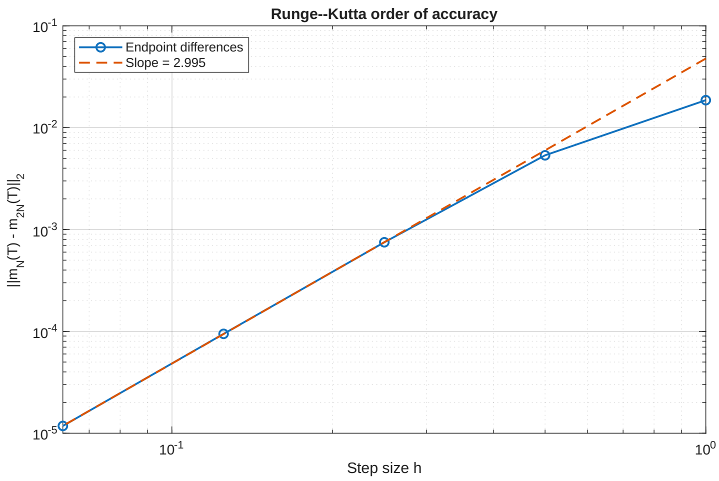

Figure 2: Endpoint differences and fitted order.

1.3 Stability limit

The eigenvalues of A are

$$\lambda_1 = 0, \qquad \lambda_{2,3} = -0.07 \pm i.$$

The stability function is

$$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6}.$$

We need $|R(h\lambda_j)| \leq 1$ for every eigenvalue. The zero eigenvalue gives $R(0) = 1$. For $\lambda = -0.07 + i$, we solve

$$|R(h\lambda)| - 1 = 0$$

with `fzero` on $[2, 2.5]$. This gives

$$\boxed{h_0 = 2.055596}.$$

The test with $h = 2$ is stable. The test with $h = 50/24 = 2.083333$ is unstable.

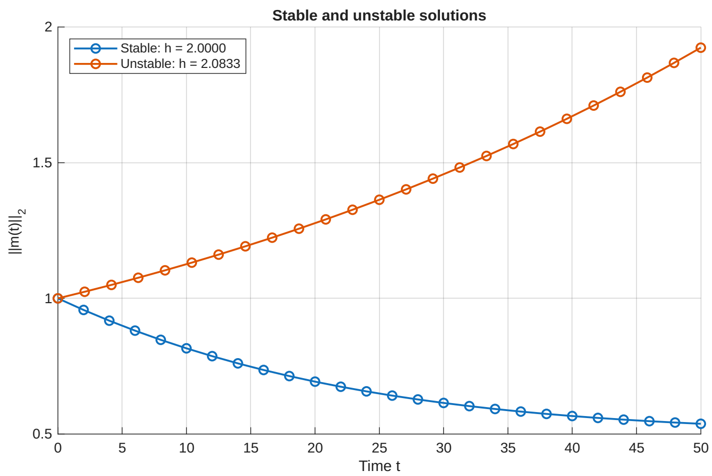

Figure 3: Stable and unstable solutions on opposite sides of h_0.

2 Robertson's stiff system

The concentrations $x = [x_A, x_B, x_C]^\mathsf{T}$ satisfy

$$x_A' = -r_1 x_A + r_2 x_B x_C, \tag{2.1}$$
$$x_B' = r_1 x_A - r_2 x_B x_C - r_3 x_B^2, \tag{2.2}$$
$$x_C' = r_3 x_B^2, \tag{2.3}$$

where

$$r_1 = 5.0 \times 10^{-2}, \qquad r_2 = 1.2 \times 10^{4}, \qquad r_3 = 4.0 \times 10^{7}, qquadx(0) = \begin{bmatrix}1 & 0 & 0\end{bmatrix}^{\mathsf{T}}.$$

The sum $x_A + x_B + x_C$ should stay equal to 1, and it did so in our results.

2.1 Explicit Runge–Kutta method

On $0 \le t \le 10$, $h = 7.0 \times 10^{-4}$ gave a stable solution. In the normal plot, x_B is too small to see clearly. The log plot shows its fast change near $t = 0$.

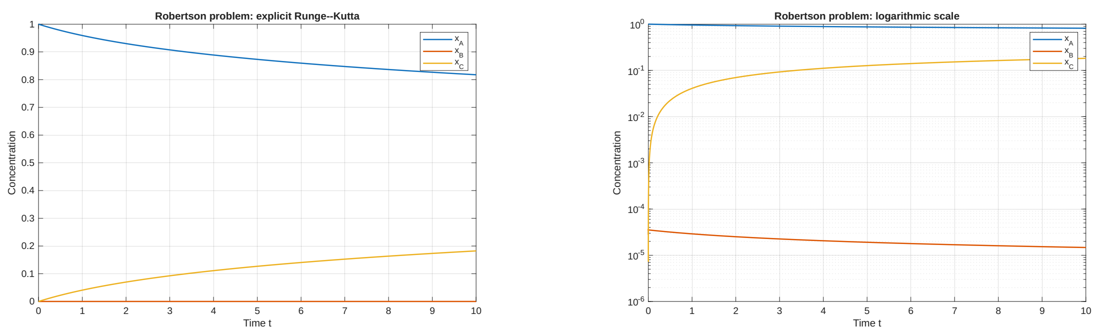

Figure 4: Explicit RK solution on $0 \le t \le 10$ with $h \approx 7.0 \times 10^{-4}$.

The Jacobian is

$$J(x) = \begin{bmatrix} -r_1 & r_2 x_C & r_2 x_B \\ r_1 & -r_2 x_C - 2 r_3 x_B & -r_2 x_B \\ 0 & 2 r_3 x_B & 0 \end{bmatrix}.$$

For a small change e, the linearized equation is $e' \approx J(x(t))e$. Therefore we check $|R(h\lambda_j(t))| \le 1$. One eigenvalue is zero. Along the computed solution,

$$\max_t |\lambda_j(t)| = 3365.924700.$$

The end of the RK stability region on the negative real axis gives

$$h_{\max} \approx 7.4652 \times 10^{-4},$$

which is close to the value found by testing. The large eigenvalue grows with time, so the allowed step size gets smaller.

For the long run, we stored only the current value of x.

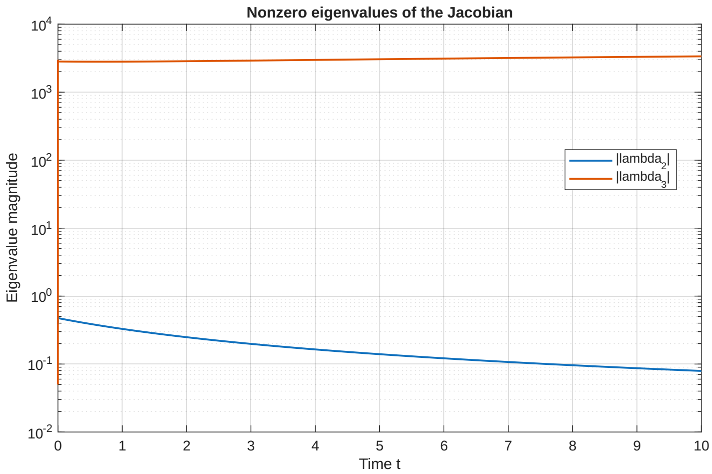

Figure 5: Magnitudes of the two nonzero Jacobian eigenvalues.

Table 1: Explicit RK result at $t = 1000$.

Quantity	Value
h	2.8×10^{-4}
$x_A(1000)$	0.293414227164
$x_B(1000)$	$1.716342048384 \times 10^{-6}$
$x_C(1000)$	0.706584056494
Time	2.93 s

2.2 Implicit Euler

Implicit Euler satisfies

$$u_{n+1} = u_n + hf(u_{n+1}).$$

At each step, the supplied function solves

$$F(v) = v - hf(v) - u_n = 0$$

with Newton's method. Here v is the new solution value:

$$(I - hJ_f(v))d = -F(v), \qquad v \leftarrow v + d.$$

Runs with $h = 10$, 1 and 0.1 did not blow up. Large steps are stable, but not accurate. With $h = 0.1$, the IE and RK curves are close on $[0, 10]$. We used the same step size on $[0, 1000]$.

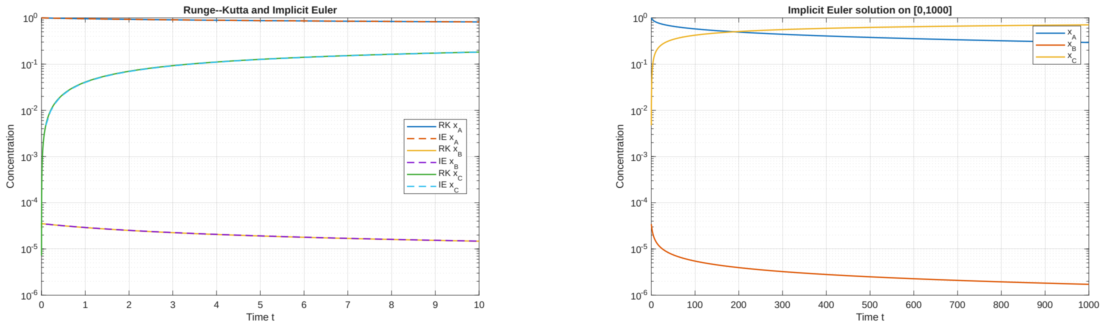

Figure 6: Short-time RK/IE comparison and the IE solution on $[0, 1000]$.

2.3 Efficiency

We used the given reference value at $t = 1000$ to calculate the errors. For the timing test, the IE code stores only the current value.

IE is faster when an error around 10^{-4} or 10^{-5} is enough, since it can take large steps. For smaller errors, IE needs many steps because it is only first order. Each step also needs a Newton solve. RK is faster below an error of about 10^{-6}. Part 1 is not stiff, so IE has no useful stability benefit there. The higher-order RK method is a better choice for that problem.

Table 2: Endpoint error and single-run computational time.

Method	h	Error	Time (s)
RK	2.8×10^{-4}	3.06×10^{-13}	2.93
IE	1	4.93×10^{-4}	0.014
IE	0.1	5.03×10^{-5}	0.048
IE	0.01	5.04×10^{-6}	0.409
IE	0.001	5.04×10^{-7}	4.04

3 One-dimensional convection–diffusion equation

The problem is

$$-T''(z) + vT'(z) = Q(z), \qquad 0 < z < 1,$$

with $T(0) = T_0$ and

$$-T'(1) = \alpha(v)(T(1) - T_{\text{out}}), \qquad \alpha(v) = \sqrt{\frac{v^2}{4} + \alpha_0^2} - \frac{v}{2}.$$

The parameters are

$$a = 0.1,\ b = 0.4,\ Q_0 = 7000,\ \alpha_0 = 50,\ T_{\text{out}} = 25,\ T_0 = 100,$$

and

$$Q(z) = \begin{cases} Q_0 \sin\left(\dfrac{(z-a)\pi}{b-a}\right), & a \le z \le b, \\ 0, & \text{otherwise.} \end{cases}$$

Let $z_j = jh$, $h = 1/N$, and $U = [T_1, \dots, T_N]^{\mathsf{T}}$. At $j = 1, \dots, N-1$,

$$\ell T_{j-1} + dT_j + rT_{j+1} = Q(z_j),$$

where

$$\ell = -\frac{1}{h^2} - \frac{v}{2h}, \qquad d = \frac{2}{h^2}, \qquad r = -\frac{1}{h^2} + \frac{v}{2h}.$$

The known value T_0 is moved to the first entry on the right-hand side. We use this second-order row at $z = 1$:

$$T_{N-2} - 4T_{N-1} + (3 + 2h\alpha)T_N = 2h\alpha T_{\text{out}}.$$

We solve the sparse matrix system with Matlab backslash.

3.1 Grid refinement

Table 3: Temperature at the exact grid point $z = 0.5$.

N	h	$T_h(0.5)$
80	0.0125	258.430491144194
160	0.00625	258.621578905525
320	0.003125	258.669336283685

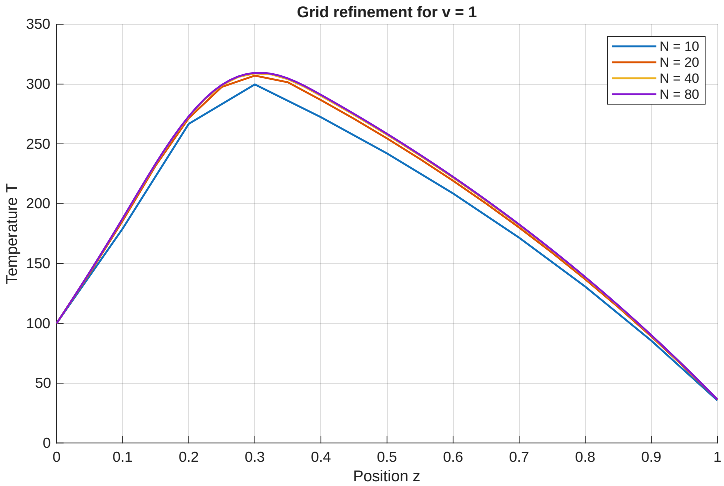

Figure 7: Solutions for $v = 1$ and $N = 10, 20, 40, 80$.

The three values give

$$p = \log_2\left(\frac{|T_{80}(0.5) - T_{160}(0.5)|}{|T_{160}(0.5) - T_{320}(0.5)|}\right) = \boxed{2.000440}.$$

This is very close to order 2. We evaluate Q at the grid points z_j and read $T(0.5)$ at $j = N/2$ on every grid.

3.2 Different velocities

Table 4: Grids used for the velocity comparison.

v	N	h
1	80	0.0125
5	100	0.0100
15	300	0.003333
100	2000	0.0005

When v is larger, the fluid moves the heat farther down the pipe. The temperature rise becomes smaller and the curve becomes flatter. The change near $z = 1$ also gets sharper, so a smaller h is needed to avoid oscillations.

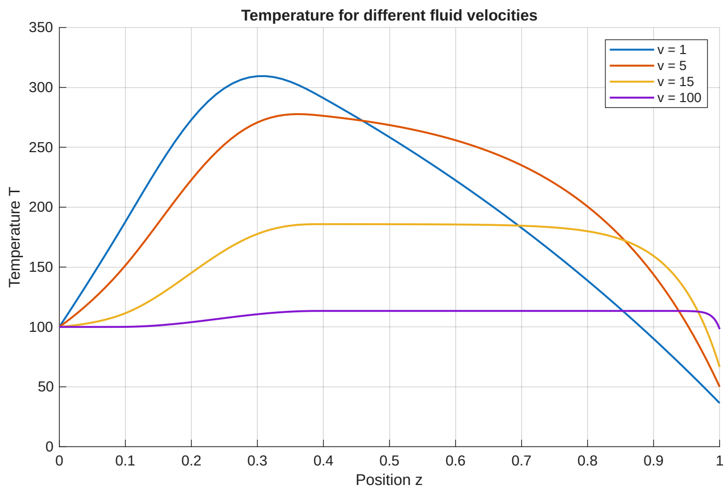

Figure 8: Temperature profiles for $v = 1, 5, 15, 100$.

4 Two-dimensional elliptic heat equation

The problem is

$$-\Delta T = f \quad \text{in } (0,12) \times (0,5),$$

with $T(x,0) = T_{\text{ext}} = 25$. The left, right and top sides are insulated, so the derivative across each side is zero. The mesh has

$$h = \frac{12}{N} = \frac{5}{M}.$$

The values at the bottom are known. The other grid points, with $i = 0, \ldots, N$ and $j = 1, \ldots, M$, are numbered by

$$k = i + 1 + (j-1)(N+1).$$

Each numbered point gives one row in the matrix. The matrix size is therefore $(N+1)M \times (N+1)M$. At an inner grid point,

$$4T_{i,j} - T_{i-1,j} - T_{i+1,j} - T_{i,j-1} - T_{i,j+1} = h^2 f(x_i, y_j).$$

For $j = 1$, the known lower value adds T_{ext} to the right-hand side. The second-order boundary rows are

$$\begin{aligned} -3T_{0,j} + 4T_{1,j} - T_{2,j} &= 0, \\ 3T_{N,j} - 4T_{N-1,j} + T_{N-2,j} &= 0, \\ 3T_{i,M} - 4T_{i,M-1} + T_{i,M-2} &= 0. \end{aligned}$$

At the two upper corners we use the side rows. We now have one equation for each unknown.

4.1 Constant source

For $f \equiv 2$ and $h = 0.2$, the computed value is

$$\boxed{T_h(6,2) = 41.000000000000}.$$

Put $T = c_0 + c_1 y + c_2 y^2$ into the PDE and the boundary conditions. This gives

$$-2c_2 = 2, \qquad c_0 = 25, \qquad c_1 + 10c_2 = 0.$$

Hence

$$\boxed{T(x,y) = 25 + 10y - y^2}, \qquad T(6,2) = 41.$$

A Taylor expansion gives

$$\frac{g(x+h) - 2g(x) + g(x-h)}{h^2} = g''(x) + \frac{h^2}{12} g^{(4)}(x) + O(h^4).$$

For a quadratic, $g^{(3)} = g^{(4)} = 0$. The inner stencil and the one-sided boundary formulas are then exact. The difference at $(6,2)$ was only 3.70×10^{-13}, which comes from rounding in the computer.

4.2 Localized source

For

$$f(x,y) = 100 \exp\left(-\frac{1}{2}(x-4)^2 - 4(y-1)^2\right),$$

we build the matrix in the same way. Only the right-hand side changes.

The successive-difference estimate is

$$\boxed{p = 2.059792},$$

which is close to order 2. The finest grid has 24100 unknowns. A sparse matrix saves both memory and time because most entries are zero.

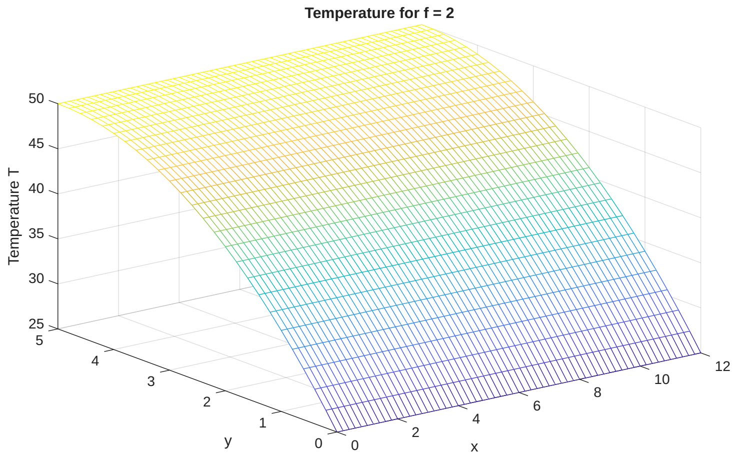

Figure 9: Numerical solution for $f \equiv 2$ and $h = 0.2$.

Table 5: Localized-source values at $(6, 2)$.

h	N	M	$T_h(6, 2)$
0.20	60	25	47.219954048124
0.10	120	50	47.224023288355
0.05	240	100	47.224999298104

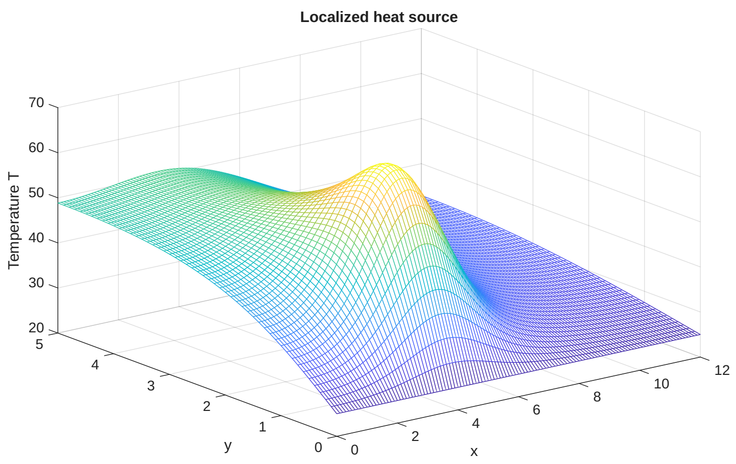

Figure 10: Mesh plot for the localized source with $h = 0.1$.

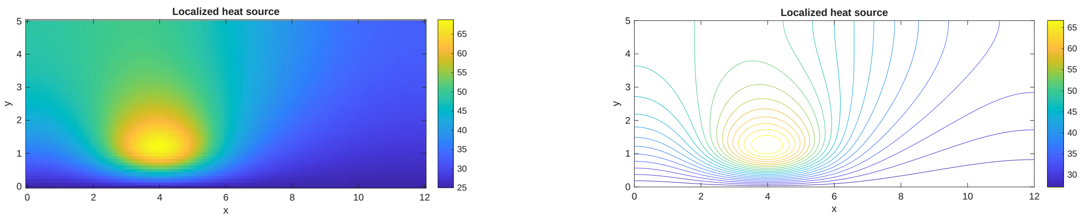

Figure 11: Image and contour plots for $h = 0.1$. The hottest area is close to the source at $(4, 1)$.

5 Conclusions

The RK method had order 3, and the stability test matched the value from the stability region. For the stiff Robertson problem, IE was faster for larger errors, while RK was faster when a small error was needed. The 1D and 2D finite difference methods both had order 2. In Part 4, the numerical solution was exact for the quadratic test problem, apart from rounding errors.

Submitted MATLAB files

- `ce1_part1_rk_landau_lifshitz.m`, `rk3_step.m`
- `ce1_part2_robertson.m`, `impeuler.m`, `impeuler_nosave.m`
- `ce1_part3_convection_diffusion_1d.m`, `solve_convection_diffusion.m`
- `ce1_part4_elliptic_heat_2d.m`, `solve_heat_2d.m`